

Système de stockage distribué basé sur une DHT

Projet Master Systèmes et Applications Répartis (SAR)

PSAR – Sorbonne Université

Auteurs: Cedric Genevieve, Safa Gunes, Somaya Boudalil
Étudiants en Master 2026 — Systèmes et Applications Répartis

Instructeur: Prof. Alessio Pagliari

Date: 15 mars 2026

Table des matières

1	Analyse Comparative des Paradigmes Architecturaux de Stockage Distribué	3
1.1	Étude des approches P2P structurée: Les DHTs	3
1.1.1	Introduction au problème du Lookup	3
1.1.2	Comparaison des stratégies de recherche	4
1.1.3	Fondamentaux des Distributed Hash Tables	6
1.1.4	Mécanismes des DHTs	7
1.1.5	Interfaces des DHTs	8
1.1.6	Conclusions	8
1.2	Étude de l'approche hiérarchisée : HBase	9
1.2.1	Introduction au modèle hiérarchisé	9
1.2.2	Fondamentaux architecturaux de HBase	9
1.2.3	Mécanismes internes : partitionnement et gestion des pannes	9
1.2.4	Avantages et limites	9
1.3	Conclusion et justification du choix architectural	10
2	Méthode de répartition des données	11
2.1	Motivation et définition de la répartition des données	11
2.2	Les différentes méthodes de répartitions des données	11
2.2.1	hachage classique	12
2.2.2	Hachage cohérent (Consistent Hashing)	13
2.3	Conclusion sur les méthodes de répartitions des données	17
3	Chord	17
3.1	Design du système et philosophie	17
3.2	Consistent Hashing et l'Identifiant Circle	18
3.3	Localisation de clés scalable : Finger Tables	18
3.4	Théorèmes	19
3.5	Algorithmes des nœuds	20
3.6	Algorithmes de la Finger Table	21
4	Pastry	23
4.1	Architecture générale	23
4.2	Identifiants et espace de clés	23
4.3	Structures de routage	24
4.3.1	1. Routing Table	24
4.3.2	2. Leaf Set (ensemble des feuilles)	24
4.3.3	3. Neighbor Set (ensemble des voisins)	24
4.4	Processus de routage	24
4.5	Algorithmes des nœuds et maintenance	24
5	Architecture du système et cycle de vie d'une requête	26
5.1	Choix architecturaux et problématique	26
5.2	Architecture générale basée sur Chord	26

5.2.1	Couche Overlay et Couche applicative	26
5.2.2	Cycle de vie d'une requête	26
5.2.3	Conclusion sur l'architecture Chord	27
5.3	Architecture générale basée sur Pastry	27
5.3.1	Couche overlay et couche applicative dans Pastry	27
5.3.2	Cycle de vie d'une requête dans Pastry	28
5.3.3	Conclusion sur l'architecture Pastry	28
6	Première implémentation triviale d'un DHT	29
6.1	L'Entité de Base : Du Nœud au Thread	29
6.2	Le Mécanisme de Communication : La Mailbox	30
6.3	Évolution du Message : Du simple "Sender" à l'"Origin"	30
6.4	Mesure du parcours et topologie du système	30
6.5	Analyse des Erreurs et Limites de la V0	31
6.6	Erreurs majeures rencontrées	31
6.7	Points d'améliorations pour la V1	31

1 Analyse Comparative des Paradigmes Architecturaux de Stockage Distribué

La croissance du volume de données a rendu les architectures centralisées insuffisantes. Les systèmes distribués sont devenus la norme, mais posent un défi central : garantir un *lookup* efficace, la *consistency* et la *fault tolerance* dans un environnement de nœuds peu fiables.

Dans ce projet Master, nous comparons deux architectures de référence pour le stockage distribué, basées sur des paradigmes différents :

1. L'architecture P2P structurée (Les DHTs) Paradigme décentralisé (architecture *symmetric*) : les nœuds ont un rôle homogène et s'auto-organisent dans un *overlay* structuré. Nous considérons plusieurs DHT (Chord, CAN, Pastry, Kademlia) qui résolvent le *lookup* en général en $O(\log N)$, avec des compromis différents entre topologie logique, latence de routage, taille des tables d'état et résilience au *churn*.

2. L'architecture hiérarchisée orientée colonnes (Modèle HBase) À l'opposé du P2P pur, ce paradigme suit une architecture *master-worker* inspirée de Bigtable. HBase vise une cohérence forte, un modèle orienté colonnes/versionné, et conserve une centralisation partielle (HMaster) pour la gestion des métadonnées et des transactions, en échange de meilleures performances d'écriture et de scan. La suite du document présente d'abord le routage et la maintenance des DHTs, puis l'architecture interne de HBase et ses mécanismes de persistance.

1.1 Étude des approches P2P structurée: Les DHTs

Cette section résume les principes essentiels des *Distributed Hash Tables* (DHTs) : localisation des données, routage, gestion des nœuds et interfaces applicatives.

Référence : DBLP:conf/p2p/WehrleGR05.

1.1.1 Introduction au problème du Lookup

Le problème central est le *lookup problem* : où stocker une donnée D , puis comment la retrouver dans un réseau distribué sans coordination centrale.

"Où stocker, et comment trouver un certain élément de donnée D dans un système distribué sans aucun contrôle ou coordination centralisés ?"

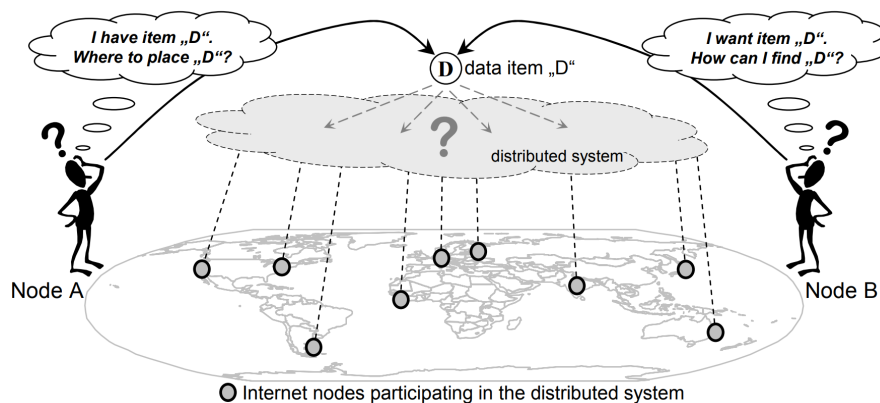


FIGURE 1 – **Le problème du lookup** : le nœud A stocke une donnée D et le nœud B doit la retrouver sans connaître son emplacement.

1.1.2 Comparaison des stratégies de recherche

Trois familles de solutions sont comparées.

1. Central Server

Approche de **première génération** (ex. Napster). Un serveur central indexe les contenus. Recherche rapide en $O(1)$, mais dépendance forte à un *single point of failure*.

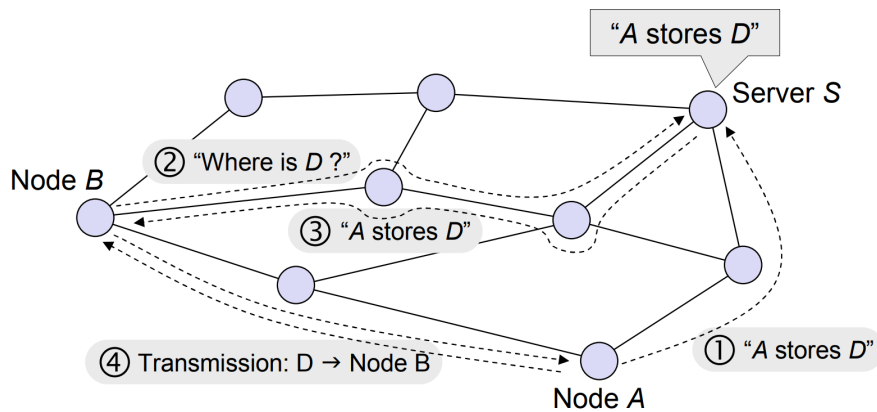


FIGURE 2 – **Central Server** : publication, requête au serveur central, puis téléchargement direct entre pairs.

2. Flooding Search

Approche de **deuxième génération** (ex. Gnutella). La requête est propagée de voisin en voisin via un *reactive flooding* jusqu'au *Time To Live* (TTL). L'approche évite le serveur central, mais génère un fort *communication overhead* et des résultats non garantis.

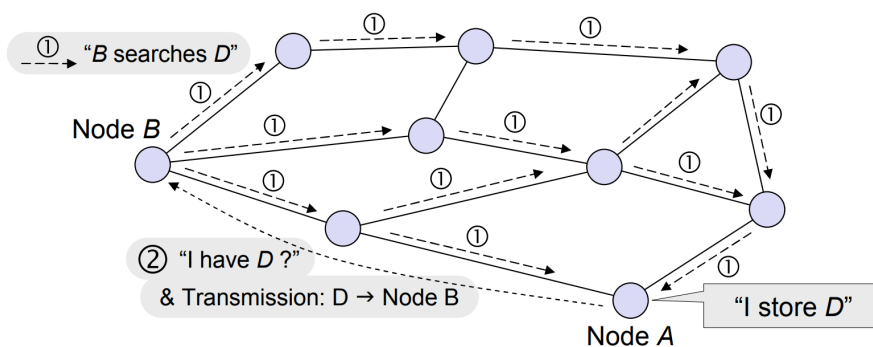


FIGURE 3 – **Flooding Search** : diffusion récursive de requêtes, puis retour des nœuds ayant la donnée.

3. Distributed Hash Tables (DHTs)

Les DHTs (*structured Peer-to-Peer systems*) structurent l'*overlay* pour obtenir un compromis robuste : $O(\log N)$ entrées de routage par nœud et $O(\log N)$ hops pour localiser une donnée.

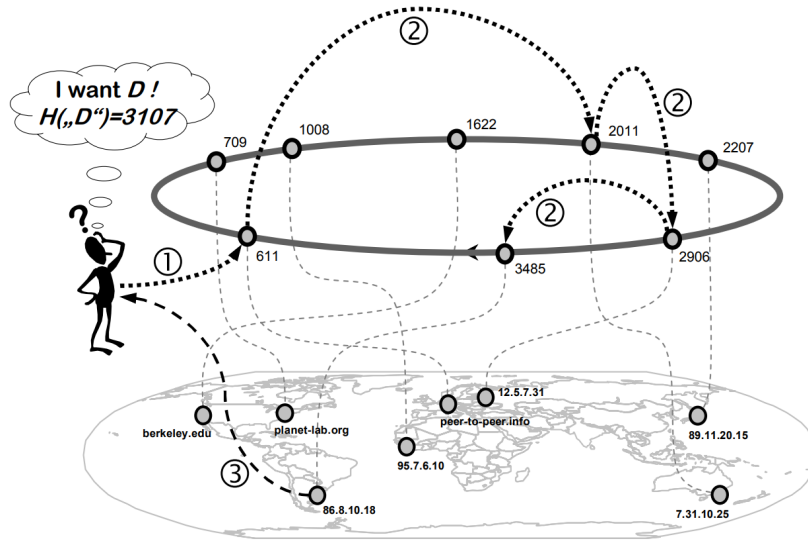


FIGURE 4 – **Distributed Hash Table** : routage structuré et efficace vers le nœud responsable.

Les Figures 2, 3, 4 et 5 montrent que les DHTs offrent le meilleur compromis entre coût de recherche et coût de stockage.

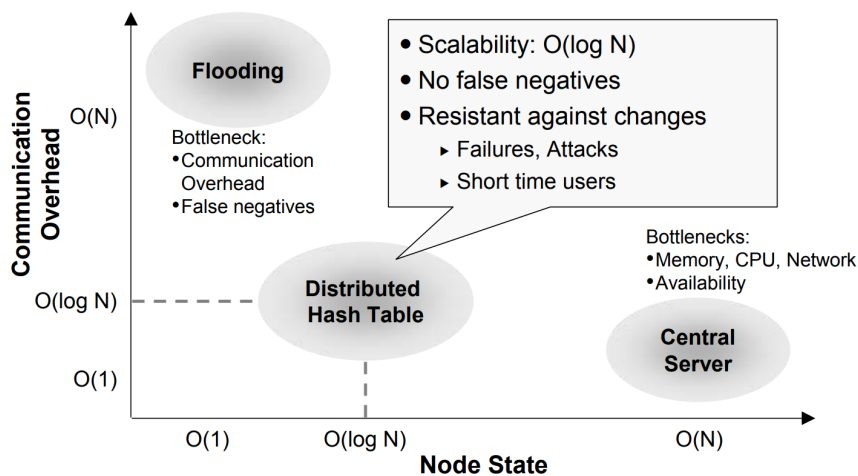


FIGURE 5 – Comparaison de la complexité en termes d'effort de recherche (axe y) et de coût de stockage par nœud (axe x). Les goulots d'étranglement (*bottlenecks*) ainsi que les caractéristiques spécifiques de chaque approche sont indiqués.

1.1.3 Fondamentaux des Distributed Hash Tables

Une DHT associe chaque clé à un nœud responsable via un espace d'adressage logique et un routage déterministe.

Gestion distribuée des données

Chaque nœud gère une plage de clés et conserve une *partial view* du système. Les messages sont relayés de nœud en nœud jusqu'à la destination.

Adressage dans les Distributed Hash Tables

Nœuds et données partagent le même *address space* logique (souvent très grand, p. ex. 0 à $2^{160} - 1$). Les IDs sont produits par une *collision-resistant hash function*.

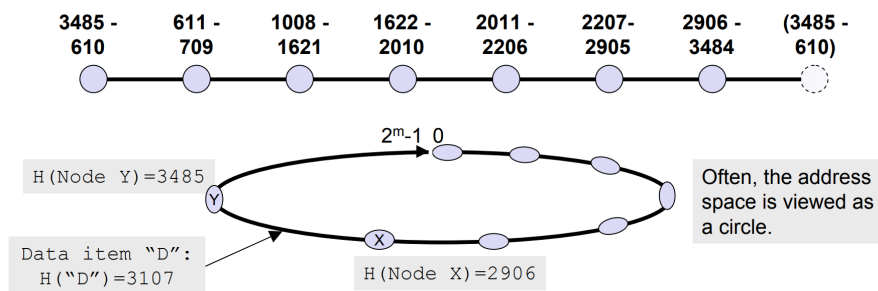


FIGURE 6 – Espace d'adressage linéaire partitionné entre pairs.

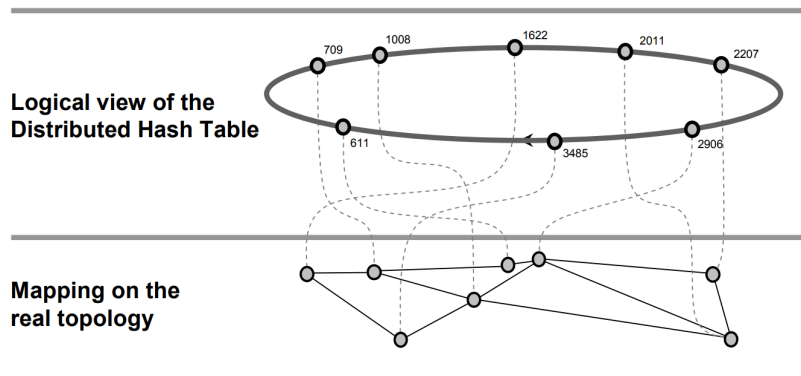


FIGURE 7 – Vue logique de la DHT et vue du réseau sous-jacent.

Routage

Chaque nœud transmet vers un voisin plus proche de l'ID cible, ce qui réduit progressivement la distance jusqu'au nœud responsable en un *small number of hops*.

Méthodes de stockage

Direct Storage : la donnée est copiée sur le nœud responsable (meilleure *availability*, coût plus élevé).

Indirect Storage : la DHT stocke un *pointer* vers la donnée (coût réduit, dépendance au nœud source).

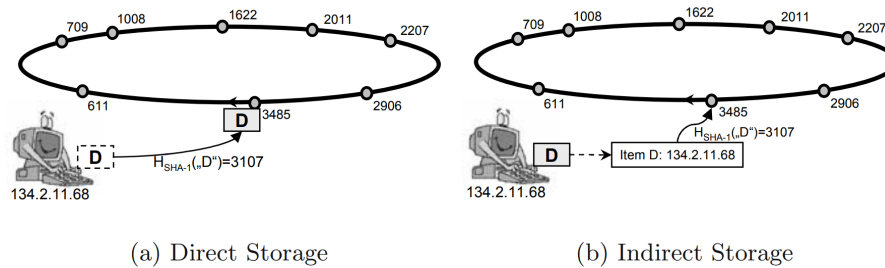


FIGURE 8 – Comparaison entre stockage direct et stockage indirect.

1.1.4 Mécanismes des DHTs

La robustesse dépend de la gestion du *churn*, c'est-à-dire le taux de *joins* et *leaves* (arrivées/départs) ainsi que les pannes fréquentes.

Arrivée de nœuds

L'entrée d'un nœud suit quatre étapes courtes :

1. **Bootstrap** : le nouveau pair contacte un nœud existant qui sert d'*entry point*.
2. **Address Assignment** : il reçoit sa partition dans le *logical address space*.
3. **Routing Update** : les tables de routage sont ajustées pour intégrer le nouveau pair.
4. **Data Transfer** : les paires (*key*, *value*) devenues sa responsabilité lui sont transférées.

Défaillance de nœuds

Deux approches coexistent : *reactive recovery* (détection pendant le routage, puis route alternative) et *proactive recovery* (*probes* périodiques, remplacement préventif des entrées invalides). La *replication* limite la perte de données applicatives.

Départ de nœuds

Un départ volontaire (*graceful leave*) facilite la migration immédiate des données vers le *successor* et accélère la convergence des tables de routage. Il améliore aussi la *routing efficiency* et stabilise les mécanismes de *replication* et de *load-balancing*.

1.1.5 Interfaces des DHTs

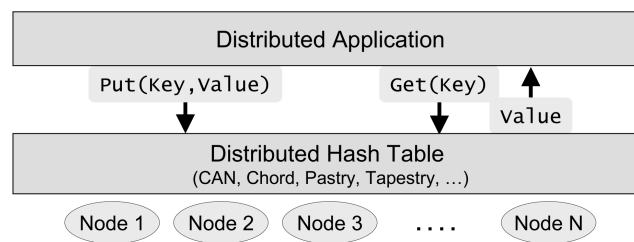


FIGURE 9 – Interface d’une Distributed Hash Table, typiquement put/get.

Interface de routage

Interface *stateless* orientée messages : *send* livre vers un ID destination, *receive* remet les messages à l’application (*delivery semantics*). Cette *stateless interface* se concentre sur la livraison de paquets basés sur un ID logique plutôt que sur une adresse IP physique.

Interface de stockage

Interface clé-valeur : *put* écrit la donnée, *get* la récupère. La couche gère *routing failures*, *replication* et *admission control*. Cette couche ajoute de la complexité car elle gère la persistance et la fiabilité des données, comme une table de *hash* distribuée.

Interface Client

Des hôtes peuvent consommer le service DHT sans participer au routage de l’*overlay* (mode *client*). Ce mode est utile quand on veut séparer usage applicatif et participation au routage (fiabilité, politiques d’*access control*).

1.1.6 Conclusions

Les DHTs offrent une excellente *scalability* sans point central, mais imposent des compromis entre efficacité de routage, coût de maintenance et résilience au *churn*.

Défis de conception

- **Routing Efficiency** : qualité du routage selon la structure des tables et la topologie.
- **Management Overhead** : coût de maintenance en continu.
- **Churn** : impact des arrivées/départs simultanés sur la stabilité.

Limitations fondamentales

- **Untrusted Environments** : résistance difficile face aux nœuds malveillants et aux *Byzantine faults*.
- **Query Constraints** : *fuzzy searches* et requêtes textuelles complexes peu adaptées au modèle par identifiants.

1.2 Étude de l'approche hiérarchisée : HBase

Les DHT répondent bien au *lookup* en environnement dynamique, mais certaines applications exigent davantage de contrôle sur la cohérence, la persistance et les performances de scan/écriture. L'approche hiérarchisée d'Apache HBase répond à ce besoin.

1.2.1 Introduction au modèle hiérarchisé

HBase, construit sur HDFS (*Hadoop Distributed File System*), est une base NoSQL distribuée orientée grands volumes. Contrairement aux DHT décentralisées, HBase suit un modèle *master-worker* avec *range partitioning* : adressage déterministe par *row key*, scans séquentiels efficaces et cohérence forte au niveau ligne.

1.2.2 Fondamentaux architecturaux de HBase

Le *HBase Master* gère le cluster (allocation des régions, équilibrage, détection de pannes), tandis que les *RegionServers* stockent les données et exécutent les lectures/écritures. Cette séparation simplifie l'administration et le contrôle opérationnel.

Modèle de données et espace d'adressage

Le modèle est orienté colonnes (tables, *rows*, familles de colonnes). Les *row keys* ordonnées lexicographiquement définissent l'espace d'adressage : excellent pour les scans et les requêtes par plages, mais sensible au design des clés.

1.2.3 Mécanismes internes : partitionnement et gestion des pannes

Deux mécanismes portent la scalabilité et la durabilité : partitionnement par régions et réaffectation après panne.

Partitionnement horizontal par plages de clés

Les tables sont découpées en régions couvrant des plages continues de *row keys*. Avec la croissance des données, *region splitting* répartit la charge entre *RegionServers*. Limite clé : des clés monotones (ex. timestamps) créent des *hotspots* et du *data skew* (*storage skew/access skew*).

Par rapport au *consistent hashing* des DHT (distribution pseudo-aléatoire), HBase choisit un partitionnement déterministe par ordre de clés.

Gestion des pannes et réaffectation

En cas de panne d'un *RegionServer*, le Master (avec ZooKeeper) réassigne les régions. Avec le *WAL* et HDFS, la reconstruction est robuste. Ce modèle est centralisé, contrairement à la réorganisation incrémentale des DHT.

1.2.4 Avantages et limites

Avantages. Cohérence forte en lecture/écriture, très bonnes performances de scans séquentiels, et bonne scalabilité horizontale via *region splitting* + ajout de *RegionServers*.

Limites. Forte sensibilité au schéma de clés (risque de *hotspot* et *data skew*) et complexité opérationnelle élevée (HDFS, ZooKeeper, tuning et supervision du cluster).

1.3 Conclusion et justification du choix architectural

DHT et HBase répondent au même objectif (stockage distribué à grande échelle) avec des choix opposés : décentralisation + *consistent hashing* d'un côté, coordination centralisée + *range partitioning* de l'autre.

Critère d'évaluation : gestion du data skew

Le *data skew* est déterminant : HBase peut concentrer charge/stockage si les clés sont mal conçues, alors que les DHT réduisent structurellement ce risque via une distribution pseudo-aléatoire des identifiants.

Motivation du choix

Notre objectif est d'implémenter un système totalement décentralisé en anneau logique. Les DHT sont mieux alignées avec cet objectif (routage distribué, auto-organisation, joins/leaves, absence de maître), tandis qu'une approche HBase déplacerait le projet vers l'ingénierie d'un cluster hiérarchisé.

Conclusion du choix

HBase reste excellent pour des besoins de cohérence forte et de scans massifs, mais pour nos objectifs pédagogiques/expérimentaux, l'approche DHT est plus pertinente. En conséquence, nous retenons l'architecture DHT comme base de la suite du projet.

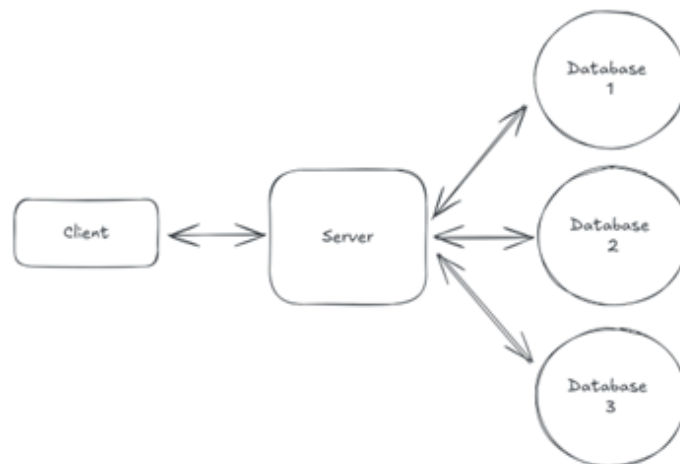
2 Méthode de répartition des données

2.1 Motivation et définition de la répartition des données

Avec l'augmentation continue du volume de données et du nombre d'utilisateurs, les architectures de stockage centralisées atteignent rapidement leurs limites. Une base de données unique n'a plus la capacité de contenir toute ces données et constitue un point de défaillance susceptible de compromettre la disponibilité du système.



Une solution pour pallier à ces limitations consiste à répartir les données et les requêtes sur plusieurs bases de données, afin de distribuer la charge de stockage et de traitement, et d'éviter la surcharge d'une base de données unique.



Toutefois, cette distribution soulève un problème fondamental : il devient nécessaire de déterminer de manière systématique sur quelle base de données une donnée doit être stockée et comment la retrouver efficacement. C'est pour répondre à cette problématique que les méthodes de répartition des données (data partitioning ou data sharding) ont été introduites. Elles définissent les règles de placement des données ce qui impacte directement les performances et l'équilibrage de charge. Ces méthodes de répartitions des données impactent également l'évolution du système dans le temps, c'est-à-dire lors de l'ajout ou le retrait d'une base de données.

2.2 Les différentes méthodes de répartitions des données

Après avoir montré la nécessité de répartir les données dans un système distribué, la première question qu'on se pose : comment associer chaque donnée à une base de données ?

Différentes méthodes de répartition des données permettent d'associer une clé à une base de données dans un système distribué.

Dans les exemples suivant nous allons utiliser le termes nœud qui fait référence à une instance base de données et le terme évènement qui fait référence à une donnée.

2.2.1 hachage classique

Principe général du hachage classique

Une première approche naïve appelée hachage classique (naive hashing ou modulo hashing) consiste à utiliser une fonction de hachage afin de faire la correspondance entre la clé d'une donnée et la base de données dans laquelle nous allons la stocker.

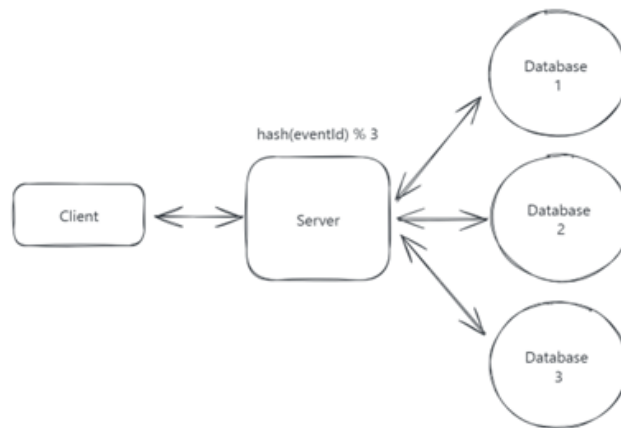
Association d'une clé à un nœud

Cette méthode repose sur les étapes suivantes :

1. Prendre l'id de l'événement et l'exécuter via une fonction de hachage, qui le convertit en nombre
2. Prendre ce nombre et effectuer l'opération modulo (%) avec le nombre de base de données.
3. Le résultat nous indique quelle base de données doit stocker cet évènement.

La formule serait donc :

$$\text{database_id} = \text{hash}(\text{event_id}) \bmod \text{number_of_databases}$$



Pour un exemple concret avec 3 bases de données :

Event #1234 → $\text{hash}(1234) \% 3 = 1 \rightarrow \text{Database 2}$

Event #5678 → $\text{hash}(5678) \% 3 = 0 \rightarrow \text{Database 1}$

Event #9012 → $\text{hash}(9012) \% 3 = 2 \rightarrow \text{Database 3}$

Ainsi, chaque clé est associée à une base de données unique, ce qui permet de retrouver l'emplacement d'une donnée à partir de sa clé.

Limite du hachage classique

Le premier problème survient lorsque l'on souhaite ajouter une nouvelle instance de base de données. La première solution à laquelle on pourrait penser consisterait à exécuter l'opération modulo avec 4 au lieu de 3 (Par rapport à l'exemple précédent).

$$\text{database_id} = \text{hash}(\text{event_id}) \% 4$$

La modification du nombre total de bases de données a non seulement affecté les données destinées à la nouvelle instance, mais a également entraîné une modification de la base de données associée à chaque événement précédemment stocké.

En effet, si on reprend un des exemples précédents :

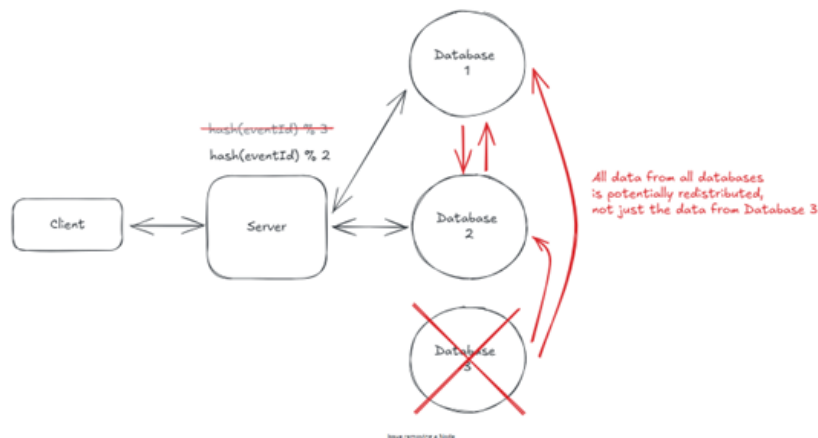
Event #1234 → $\text{hash}(1234) \% 3 = 1 \rightarrow \text{Database 2}$ $\text{hash}(1234) \% 4 = 0 \rightarrow \text{Database 1}$



L'événement n° 1234 était mappé à la base de données 2, mais avec l'ajout de la nouvelle base de données, ces données doivent donc être déplacées vers la base de données 1.

L'ajout d'une ou plusieurs bases de données entraîne donc une redistribution dans toutes les instances de base de données, ce qui provoque un mouvement massif de données inutiles. Ce mouvement massif cause d'énormes pics de charge de base de données et impact directement les utilisateurs. En effet, les utilisateurs ne pourront pas accéder à leur donnée ou auront un temps de réponse lent.

Un autre problème avec le hachage modulo simple réside dans le retrait d'une base de données. Supposons qu'une base de données soit tombée en panne. Il est nécessaire de redistribuer les données qui étaient dans la base de données tombée en panne vers les autres bases de données encore actives. Etant donné que le nombre de base de données actives diminue, le calcul pour trouver où ranger une donnée sera modifié. De ce fait, les données présentes dans les bases de données encore actives risquent d'être redistribuées vers d'autres instances de base de données.



Ces limitations montrent que le hachage modulo classique n'est pas adapté aux systèmes distribués dynamiques, où les bases de données peuvent être ajoutées ou retirées fréquemment.

2.2.2 Hachage cohérent (Consistent Hashing)

Principe général du consistent Hashing

Le consistent Hashing a été introduit pour résoudre le problème de la redistribution des données lors de l'ajout ou de la suppression d'une instance dans un système distribué. Le principe est d'organiser à la fois les données et les bases de données dans un espace circulaire appelé « anneau de hachage ».

Dans cet anneau, les clés et les nœuds sont positionnés en utilisant la même fonction de hachage. Chaque base de données se voit attribuer une position sur l'anneau, et chaque donnée est également

positionnée sur cet anneau en fonction du résultat de sa fonction de hachage.

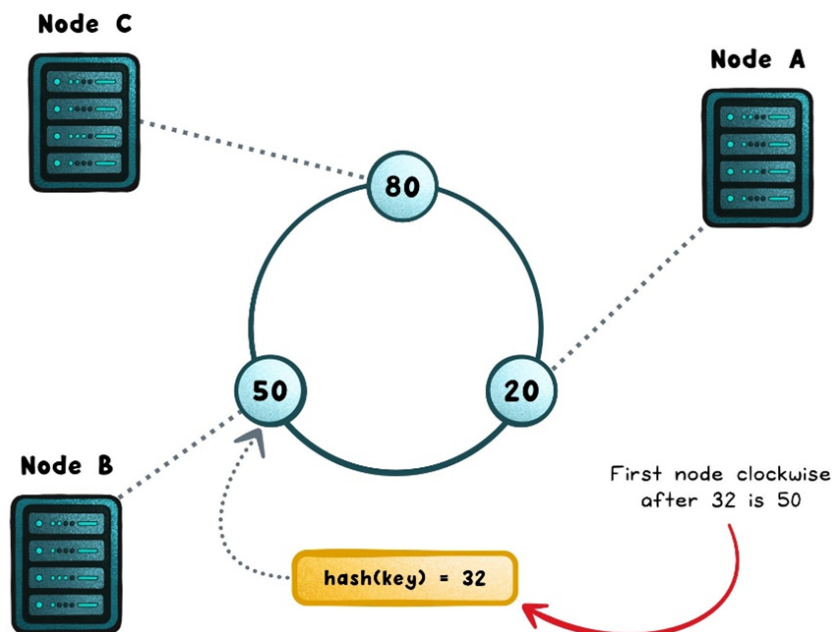
Association d'une clé à un nœud

Cette méthode repose sur les étapes suivantes :

1. On réalise d'abord un anneau de hachage avec un nombre fixe de points.
2. On positionne les bases de données sur l'anneau en appliquant la fonction de hachage à leur identifiant.
3. Pour trouver sur quelle base de données une donnée doit être stockée, on hache d'abord l'ID de la donnée comme avec le hachage classique.
4. On se déplace ensuite dans le sens des aiguilles d'une montre sur l'anneau jusqu'à rencontrer le premier nœud.
5. Ce nœud va stocker de la donnée.

Par exemple :

Pour un nombre fixe de points égal à 90, 3 bases de données positionnées aux points 20, 50 et 80. Si nous avons un $\text{hash}(\text{id_data}) = 32$, nous nous déplaçons dans le sens des aiguilles d'une montre pour trouver la première instance de base de données. Cette instance sera la base de données 2 (Node B) située au point 50.



Ajout d'une nouvelle base de données

Lorsqu'on ajoute une nouvelle base de données, seules les données qui se situent entre sa nouvelle position et la base de données précédente doivent être déplacées.

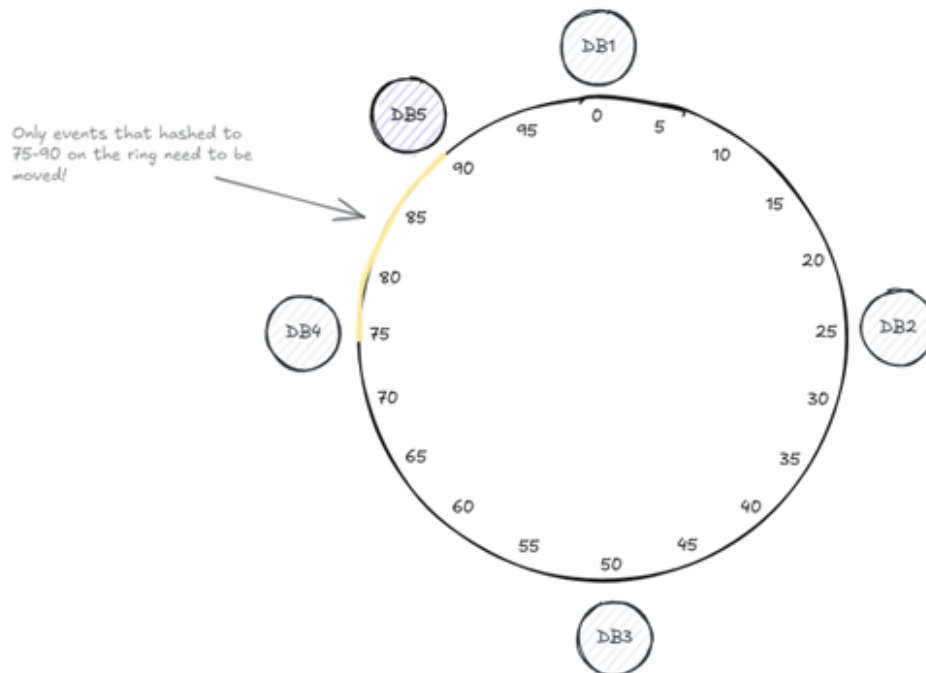
Par exemple :

Pour un nombre fixe de points égal à 100, 4 bases de données positionnées aux points 0, 25, 50 et 75.

Si nous ajoutons une cinquième base de données à la position 90 sur notre anneau :

- Seules les données qui se hachent vers des positions comprises entre 75 et 90 doivent désormais être stockées dans la base de données 5.

- Toutes les autres données restent dans leur base de données respective.



Hash Ring with DB5 added

Contrairement au hachage modulo classique, seule une petite portion des données est redistribuée.

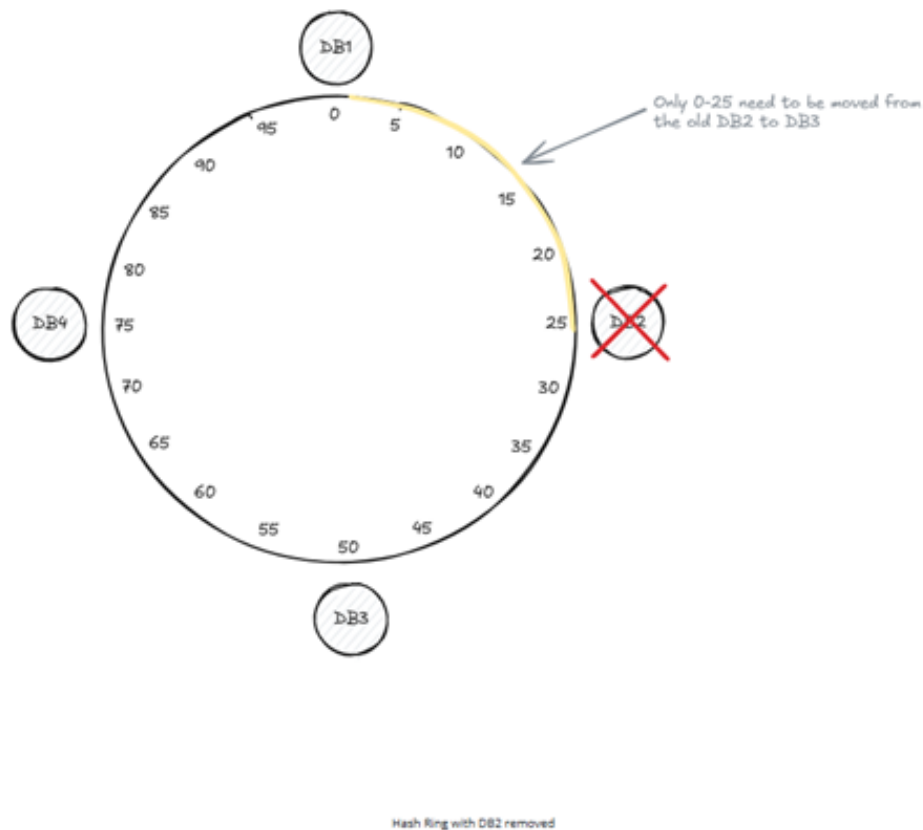
Suppression d'une base de données

En reprenant l'exemple initial :

Pour un nombre fixe de points égal à 100, 4 bases de données positionnées aux points 0, 25, 50 et 75.

Supposons que la base de données numéro 2 située à la position 25 rencontre une défaillance.

- Seules les données mappées sur la base de données 2 doivent être déplacées.
- Le déplacement se fait dans le sens des aiguilles d'une montre. Elles seront donc mappées à la base de données 3 située à la position 50.
- Toutes les autres données restent dans leur base de données respective.



Le consistent hashing permet ainsi de résoudre les principales limites du hachage classique en réduisant drastiquement le nombre de données redistribuées lors des changements de topologie (ajout ou retrait de base de données).

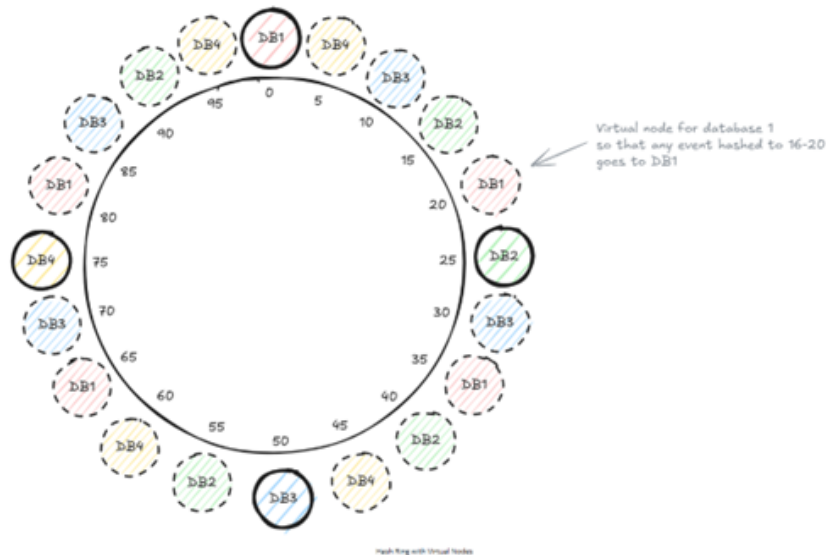
Cependant, il reste encore un problème. En effet, dans notre exemple ci-dessus où nous avons supprimé la base de données 2, nous avons dû déplacer toutes les données stockées dans la base de données 2 vers la base de données 3, ce qui peut entraîner une surcharge locale. Notre objectif serait de répartir de manière équitable les données lors de la suppression d'une base de données.

Optimisation : Nœuds Virtuels

Pour résoudre ce problème, une des solutions serait d'utiliser des « nœuds virtuels ». Au lieu de mettre chaque base de données en un seul point de sur l'anneau, nous la divisons en plusieurs points en hachant différentes variantes du nom de la base de données.

Par exemple :

« DB1 » sera divisé en « DB1-vn1 », « DB1-vn2 », « DB1-vn3 » et « DB1-vn4 » qui seront respectivement aux positions 0, 20, 35, 65, 80. C'est le même principe pour les autres bases de données.



Nous pouvons observer sur la figure ci-dessus que les nœuds virtuelles sont mélangés autour de l'anneau. Supposons que la bases de données numéro 2 rencontre une défaillance :

- Les données mappées sur « DB2-vn1 » seront stockées dans la base de données 1
- Les données mappées sur « DB2-vn2 » seront stockées dans la base de données 3
- Les données mappées sur « DB2-vn3 » seront stockées dans la base de données 4
- Et ainsi de suite

La charge de la base de données supprimé est donc répartie de manière beaucoup plus homogène dans toute les bases de données actives.

2.3 Conclusion sur les méthodes de répartitions des données

Les méthodes de répartition des données sont donc indispensables dans les systèmes distribués. En effet, elles déterminent la manière dont les données sont réparties entre les bases de données. Le hachage classique est une solution simple, mais peu adaptée lors de l'ajout ou du retrait d'une ou plusieurs bases de données, en raison des redistributions massives qu'il engendre. Le consistant hashing permet de corriger ces limitations en réduisant les déplacements de données. Une des optimisations du consistant hashing repose sur l'utilisation de nœuds virtuels, permettant d'améliorer l'équilibrage de charge entre les nœuds lors du retrait d'une base de données.

3 Chord

Chord est un protocole de *lookup* distribué qui répond à un problème fondamental : localiser efficacement le nœud stockant une donnée particulière. Alors que de nombreux systèmes *peer-to-peer* offrent des fonctionnalités complexes comme l'anonymat ou le stockage redondant, Chord se concentre exclusivement sur une opération centrale : le *mapping* d'une *key* vers un nœud.

3.1 Design du système et philosophie

Chord est conçu comme une bibliothèque liée aux applications clientes et serveurs. Il répond à cinq défis critiques des systèmes *peer-to-peer* (P2P) :

1. **Load Balance** : Chord agit comme une fonction de *hash* distribuée, répartissant les *keys* uniformément pour éviter qu'un seul nœud ne soit submergé.
2. **Decentralisation** : Aucun nœud n'est plus important qu'un autre, ce qui élimine les *single points of failure* tels que ceux rencontrés dans Napster.

3. **Scalability** : Le coût du *lookup* croît selon le logarithme du nombre de nœuds, rendant possible l'exploitation de systèmes impliquant des millions de participants.
4. **Availability** : Le système s'ajuste automatiquement aux *joins* et aux *failures*, garantissant que le nœud responsable soit toujours localisé.
5. **Flexible Naming** : Chord utilise un *flat key space*, offrant aux applications une liberté totale quant à la manière de lier leurs noms internes aux *keys* de Chord.

3.2 Consistent Hashing et l'Identifiant Circle

Chord utilise une variante du *consistent hashing* pour assigner les *keys* aux nœuds.

Les nœuds ainsi que les *keys* reçoivent un **identifiant** de m bits via une fonction de *hash* de base telle que SHA-1. L'ID d'un nœud est dérivé de son adresse IP, tandis que l'ID d'une *key* est le *hash* de la donnée elle-même.

Les identifiants sont disposés sur un cercle modulo 2^m appelé **chord ring**. Une *key* k est assignée au premier nœud dont l'identifiant est égal ou supérieur à k dans le sens des aiguilles d'une montre. Ce nœud est appelé le *successor* de la *key* k .

3.3 Localisation de clés scalable : Finger Tables

Pour éviter l'inefficacité d'un parcours linéaire du cercle, Chord maintient une table de routage appelée **finger table**.

Chaque nœud n maintient jusqu'à m entrées. La $i^{\text{ème}}$ entrée pointe vers le premier nœud qui succède à n d'au moins 2^{i-1} .

Approche par Binary Search : Cette structure permet à un nœud de stocker davantage d'informations sur les nœuds qui le suivent de près et moins sur ceux qui sont plus éloignés.

Performance : Chaque *lookup* réduit de moitié la distance vers la cible, garantissant que la destination est atteinte en $O(\log N)$ *hops*.

Les nœuds proches d'une *key* disposent d'informations de type "*fine-grained*", tandis que les nœuds plus éloignés permettent des "*long jumps*" à travers le réseau pour accélérer la recherche.

3.4 Théorèmes

L'article présente 8 théorèmes servant à justifier l'usage du *consistent hashing* et de la structure de la **finger table**.

Théorème 1

Pour tout ensemble de N nœuds et K keys, avec une probabilité élevée : chaque nœud est responsable d'au plus $(1 + \epsilon)K/N$ keys. Lorsqu'un $(N + 1)^{\text{ème}}$ nœud rejoint ou quitte le réseau, seules $O(K/N)$ keys sont déplacées.

Ce premier théorème traite de l'efficacité avec laquelle Chord répartit les données sur le réseau, réalisant un "*natural load balance*". Il garantit qu'aucune machine ne devienne un goulot d'étranglement.

Théorème 2

Avec une probabilité élevée, le nombre de nœuds devant être contactés pour trouver un successor dans un réseau de N nœuds est en $O(\log N)$.

Ce théorème démontre l'efficacité des *lookups*, même dans des systèmes comptant des millions de nœuds. Il justifie la structure de la **finger table**, où chaque *hop* est conçu pour réduire de moitié la distance vers l'ID cible.

Théorème 3

Avec une probabilité élevée, tout nœud rejoignant ou quittant un réseau Chord de N nœuds utilisera $O(\log^2 N)$ messages pour rétablir les finger tables et les invariants de routage.

Le troisième théorème quantifie le coût du maintien de la structure du réseau lors de l'arrivée ou du départ de participants. Il permet aux développeurs d'estimer le "*management overhead*" du système. Il montre que bien qu'un *join* soit plus coûteux qu'un simple *lookup*, il reste très performant à mesure que le réseau croît.

Théorème 4

Dès qu'un nœud peut résoudre avec succès une requête donnée, il sera toujours en mesure de le faire à l'avenir.

Cette propriété garantit qu'une fois que le réseau a atteint un état où un chemin vers une *key* existe, les routines de maintenance du protocole ne briseront pas cette connectivité.

Théorème 5

Passé un certain délai après le dernier join, tous les successor pointers seront corrects.

Le protocole *stabilize* garantit la convergence, ramenant ainsi le réseau vers un état de cercle parfait une fois que le "*churn*" des nœuds entrants s'arrête.

Théorème 6

Si nous prenons un réseau stable de N nœuds, et qu'un autre ensemble allant jusqu'à N nœuds rejoint le réseau sans finger pointers (mais avec des successor pointers corrects), alors les lookups prendront toujours un temps en $O(\log N)$ avec une probabilité élevée.

Cela prouve que Chord est extrêmement résilient lors de phases de croissance. Même si un nombre massif de nouveaux nœuds rejoint le réseau sans avoir encore construit leurs **finger tables**, l'infrastructure existante garantit que la vitesse de recherche ne se dégrade pas.

Théorème 7

Si nous utilisons une successor list de longueur $r = O(\log N)$ dans un réseau initialement stable, et que chaque nœud échoue avec une probabilité de $1/2$, alors avec une probabilité élevée `find_successor` renvoie le successor vivant le plus proche de la key demandée.

Ce théorème valide le mécanisme de **Successor List**. Il prouve qu'avec seulement une quantité logarithmique de redondance, le système peut localiser les données de manière fiable, même si la moitié des machines du réseau disparaissent simultanément.

Théorème 8

Si nous utilisons une successor list de longueur $r = O(\log N)$ dans un réseau initialement stable, et que chaque nœud échoue avec une probabilité de $1/2$, alors le temps attendu pour exécuter `find_successor` dans le réseau défaillant est en $O(\log N)$.

Il s'agit du pendant de performance du Théorème 7. Il garantit que non seulement la recherche trouvera le nœud correct après une défaillance de 50%, mais qu'elle le fera avec la même efficacité logarithmique.

3.5 Algorithmes des nœuds

1. L'algorithme de Successor Resolution

Le cœur de l'efficacité de Chord réside dans la fonction `find_successor`. Elle utilise la **finger table** pour effectuer des bonds exponentiels vers l'ID cible.

Algorithm: `Node.find_successor(id)`

```

1: if  $id \in (\text{node}, \text{successor}]$  then
2:   return successor
3: else
4:    $n' = \text{closest\_preceding\_node}(id)$ 
5:   return  $n'.\text{find\_successor}(id)$ 
6: end if
```

Pour trouver le nœud précédent le plus proche, le système itère à travers la **finger table**, du saut le plus grand vers le plus petit :

Algorithm: Node.closest_preceding_node(*id*)

```

1: for  $i = m$  downto 1 do
2:   if  $\text{finger}[i].\text{node} \in (\text{node}, id)$  then
3:     return  $\text{finger}[i].\text{node}$ 
4:   end if
5: end for
6: return node

```

2. Maintenance dynamique du ring : Joining et Stabilisation

Lorsqu'un nœud rejoint le réseau, il n'a besoin de connaître qu'un seul nœud existant pour s'intégrer au cercle. La fonction `stabilize` est exécutée périodiquement pour vérifier le *successor* et l'informer de l'existence du nœud.

Algorithm: Node.join(n')

```

1: predecessor = nil
2: successor =  $n'.\text{find\_successor}(\text{node})$ 

```

Algorithm: Node.stabilize()

```

1:  $x = \text{successor.predecessor}$ 
2: if  $x \in (\text{node}, \text{successor})$  then
3:   successor =  $x$ 
4: end if
5: successor.notify(node)

```

3. Résilience : Le mécanisme de Successor List

Pour gérer le "*churn*" des nœuds tombant en panne sans préavis, chaque nœud maintient une liste de r *successors*. Cela garantit que si le *successor* primaire échoue, le nœud peut toujours router vers le prochain nœud actif disponible dans la liste.

Algorithm: Node.check_predecessor()

```

1: if (predecessor has failed) then
2:   predecessor = nil
3: end if

```

Algorithm: Node.handle_successor_failure()

```

1: for each  $s \in \text{successor\_list}$  do
2:   if  $s$  is alive then
3:     successor =  $s$ 
4:     break
5:   end if
6: end for

```

3.6 Algorithmes de la Finger Table**1. Calcul des entrées de la Finger Table**

Chaque nœud n maintient une table de routage de m entrées. La $i^{\text{ème}}$ entrée est définie comme le premier nœud qui succède au point $n + 2^{i-1}$ sur le cercle d'identifiants.

$$\text{finger}[i].\text{start} = (n + 2^{i-1}) \pmod{2^m}, \quad 1 \leq i \leq m$$

$$\text{finger}[i].\text{node} = \text{find_successor}(\text{finger}[i].\text{start})$$

2. Initialisation de la Finger Table

Lorsqu'un nouveau nœud rejoint le réseau, il doit renseigner sa **finger table**. Alors qu'une approche naïve consisterait à appeler `find_successor` m fois, le protocole utilise souvent des nœuds existants pour fournir des "*hints*", réduisant ainsi la complexité de l'initialisation.

Algorithm: Node.init_finger_table(n')

```

1: finger[1].node =  $n'$ .find_successor(finger[1].start)
2: predecessor = successor.predecessor
3: successor.predecessor = node
4: for  $i = 1$  to  $m - 1$  do
5:   if finger[ $i + 1$ ].start  $\in$  [node, finger[ $i$ ].node) then
6:     finger[ $i + 1$ ].node = finger[ $i$ ].node
7:   else
8:     finger[ $i + 1$ ].node =  $n'$ .find_successor(finger[ $i + 1$ ].start)
9:   end if
10: end for

```

3. Maintenance proactive de la table

Le réseau étant dynamique, les entrées des doigts peuvent devenir "*stale*" lors de l'arrivée ou du départ de nœuds. La routine `fix_fingers` est exécutée périodiquement pour rafraîchir une entrée aléatoire, garantissant que les informations de routage demeurent *eventually consistent*.

Algorithm: Node.fix_fingers()

```

1:  $i = \text{random index} \in [1, m]$ 
2: finger[ $i$ ].node = find_successor(finger[ $i$ ].start)

```

4. Mise à jour des autres nœuds

Lorsqu'un nouveau nœud n rejoint le réseau, il peut devenir le $i^{\text{ème}}$ doigt de plusieurs nœuds existants. La fonction `update_others` identifie ces nœuds et les force à mettre à jour leurs tables pour refléter l'arrivée de n .

Algorithm: Node.update_others()

```

1: for  $i = 1$  to  $m$  do
2:    $p = \text{find\_predecessor}(\text{node} - 2^{i-1})$ 
3:    $p.\text{update\_finger\_table}(\text{node}, i)$ 
4: end for

```

Algorithm: $p.\text{update_finger_table}(s, i)$

```

1: if  $s \in [\text{node}, \text{finger}[i].\text{node})$  then
2:    $\text{finger}[i].\text{node} = s$ 
3:    $p = \text{predecessor}$ 
4:    $p.\text{update\_finger\_table}(s, i)$ 
5: end if

```

4 Pastry

Pastry est une infrastructure distribuée de localisation d'objets et de routage destinée aux systèmes pair-à-pair (P2P) à grande échelle. Elle repose sur une table de hachage distribuée (DHT), comparable à Chord, et fonctionne au-dessus d'Internet via un réseau overlay. Les nœuds logiques de Pastry ne correspondent pas nécessairement à la topologie physique du réseau IP, mais forment une structure virtuelle organisée selon l'espace des identifiants.

L'objectif fondamental de Pastry est d'acheminer efficacement un message associé à une clé donnée vers le nœud dont l'identifiant est numériquement le plus proche de cette clé. À l'instar de Chord, Pastry résout le problème du *key-based routing*, mais adopte une approche différente fondée sur un routage par préfixes plutôt que sur une structure strictement circulaire.

4.1 Architecture générale

La conception de Pastry repose sur plusieurs principes structurants :

1. Décentralisation

Aucun nœud ne possède de rôle central. Chaque participant maintient localement ses structures de routage, ce qui supprime tout point unique de défaillance.

2. Scalabilité logarithmique

Le routage s'effectue en $O(\log N)$ sauts en moyenne, où N est le nombre de nœuds du système, ce qui garantit le passage à grande échelle.

3. Auto-organisation

Le protocole s'adapte dynamiquement aux arrivées et aux départs de nœuds (*churn*) par la mise à jour progressive des structures de routage.

4. Localité réseau

Lorsque plusieurs nœuds satisfont une contrainte de routage, Pastry privilégie celui qui est le plus proche selon une métrique réseau, réduisant ainsi la latence réelle des communications.

4.2 Identifiants et espace de clés

Chaque nœud Pastry possède un identifiant unique, appelé *nodeId*, généralement codé sur 128 bits et dérivé d'une fonction de hachage cryptographique. De même, chaque message ou clé à router reçoit un identifiant correspondant dans le même espace.

L'espace d'identifiants est organisé en base 2^b , où b est un paramètre de configuration influençant la taille des tables de routage. Le routage s'effectue en recherchant le nœud actif dont le *nodeId* est numériquement le plus proche de l'identifiant cible.

Contrairement à Chord, qui dispose d'un anneau strictement circulaire, Pastry utilise un routage par préfixes, où la distance est mesurée par la longueur du préfixe commun entre identifiants.

4.3 Structures de routage

Chaque nœud Pastry maintient trois structures principales : la *Routing Table*, le *Leaf Set* et le *Neighbor Set*.

4.3.1 1. Routing Table

Organisée par préfixes d'identifiants, elle contient des pointeurs vers des nœuds partageant des préfixes communs avec le *nodeId* courant. Cette structure permet de faire des sauts logarithmiques vers la destination, rapprochant le message à chaque étape.

Cette structure est analogue aux *Finger Tables* de Chord, mais basée sur des préfixes plutôt que sur des puissances de 2.

4.3.2 2. Leaf Set (ensemble des feuilles)

Il regroupe les nœuds immédiatement supérieurs et inférieurs au *nodeId*. Il assure le routage correct pour les clés proches et fournit des alternatives en cas de défaillance de certains nœuds.

4.3.3 3. Neighbor Set (ensemble des voisins)

Il contient des nœuds physiquement proches selon la topologie réseau, indépendamment des identifiants. Il est utilisé pour optimiser la latence des communications et améliorer la localité réelle du routage.

4.4 Processus de routage

Le routage dans Pastry s'effectue en rapprochant progressivement le message de son *nodeId* cible grâce aux structures précédentes :

1. **Vérification du Leaf Set** : si la clé cible se trouve dans l'intervalle couvert, le message est envoyé au nœud le plus proche numériquement.
2. **Utilisation de la Routing Table** : si la clé n'est pas dans le Leaf Set, le message est transmis à un nœud partageant un préfixe plus long avec la clé.
3. **Sélection par proximité physique** : si aucune entrée n'est disponible ou valide, un nœud du Leaf Set numériquement plus proche est choisi, en prenant en compte la distance réseau.

À chaque étape, le message progresse vers un nœud plus proche de la destination, ce qui garantit la convergence du routage en $O(\log N)$ sauts en moyenne pour N nœuds actifs.

4.5 Algorithmes des nœuds et maintenance

Chaque nœud Pastry exécute des algorithmes pour intégrer de nouveaux nœuds, maintenir les structures de routage et gérer les défaillances.

1. **Intégration d'un nouveau nœud (*join*)**
 - Un nœud entrant contacte un nœud existant pour s'intégrer au réseau.
 - Les structures de routage (*Routing Table*, *Leaf Set*, *Neighbor Set*) sont initialisées progressivement grâce aux informations des nœuds voisins.
2. **Maintenance dynamique (*stabilize* et mise à jour du routage)**
 - Les entrées de la *Routing Table* sont périodiquement mises à jour pour refléter les départs ou arrivées de nœuds.

- Le *Leaf Set* et le *Neighbor Set* sont également ajustés pour maintenir la cohérence et la résilience locale.

3. Tolérance aux pannes

- En cas de défaillance d'un nœud, les messages sont redirigés vers le nœud le plus proche dans le *Leaf Set* ou via la *Routing Table*.
- Cette redondance garantit que le routage reste correct et efficace même avec un nombre important de nœuds défaillants.

5 Architecture du système et cycle de vie d'une requête

5.1 Choix architecturaux et problématique

Dans le cadre de notre projet, nous avons choisi d'implémenter une architecture de stockage distribuée reposant sur des protocoles de type Distributed Hash Table (DHT), et plus précisément sur Chord et Pastry. Ces deux protocoles partagent un objectif commun : organiser un ensemble de nœuds distribués de manière totalement décentralisée afin d'assurer le stockage et la localisation efficace de données dans un espace d'identifiants partagé.

Avant d'implémenter ces protocoles, il est essentiel de comprendre précisément l'architecture que nous allons construire ainsi que le protocole applicatif qui s'appuiera dessus. Autrement dit, nous devons être capables de répondre aux questions suivantes :

- À quel nœud une requête est-elle envoyée initialement ?
- Comment est déterminé le nœud responsable d'une donnée ?
- Comment les nœuds communiquent-ils entre eux ?
- Que se passe-t-il concrètement lors d'une opération élémentaire telle que PUT ou GET ?
- Quel est le cycle de vie complet d'une requête ?

Dans cette première partie, nous détaillons l'architecture retenue pour Chord, en analysant précisément la circulation des messages, la responsabilité des nœuds et le traitement des requêtes. La même démarche sera ensuite appliquée à Pastry.

5.2 Architecture générale basée sur Chord

5.2.1 Couche Overlay et Couche applicative

Il est essentiel, dans notre implémentation, de distinguer clairement deux niveaux fonctionnels qui coexistent au sein de chaque nœud.

Le premier niveau correspond à la couche overlay, c'est-à-dire le protocole Chord lui-même. Cette couche est responsable de l'organisation logique du réseau en anneau, du maintien des pointeurs (successeur, prédécesseur, finger table) et du routage des messages. Son rôle est exclusivement de répondre à la question suivante : quel nœud est responsable d'un identifiant donné ? Autrement dit, Chord assure la localisation distribuée des clés.

Le second niveau correspond à la couche applicative, que nous implémentons au-dessus de Chord. Cette couche gère concrètement le stockage des paires (clé, valeur) sur le nœud responsable. Elle implémente les opérations élémentaires telles que PUT, GET ou DELETE, et décide éventuellement des mécanismes de réplication ou de gestion de la cohérence.

Ainsi, Chord ne stocke pas directement les données : il fournit un mécanisme de routage déterministe permettant d'identifier le bon nœud. La gestion effective des données repose sur la couche applicative locale de chaque nœud.

5.2.2 Cycle de vie d'une requête

Requête PUT

Lorsqu'un client souhaite insérer une donnée via PUT(key, value), la requête est envoyée à un nœud arbitraire du réseau. Ce nœud devient le point d'entrée du système, mais il n'est pas nécessairement responsable de la clé.

Le nœud calcule d'abord l'identifiant associé à la clé en appliquant la fonction de hachage. L'identifiant obtenu est positionné dans le même espace circulaire que les nœuds.

Le nœud exécute alors la procédure `find_successor(id)`. Si l'identifiant appartient à l'intervalle compris entre le nœud courant et son successeur, alors ce successeur est responsable. Sinon, le nœud consulte sa finger table pour déterminer le `closest preceding node`, c'est-à-dire le nœud le plus proche

précédant la cible. La requête est alors transmise à ce nœud. Ce processus se répète récursivement jusqu'à atteindre le nœud responsable.

Une fois le nœud responsable localisé, la paire (clé, valeur) est stockée localement. Pour assurer une tolérance aux pannes, cette donnée peut être répliquée sur plusieurs successeurs afin d'assurer une redondance. Une réponse de confirmation est ensuite renvoyée au client.

Requête GET

Le traitement d'une requête GET(key) suit exactement la même logique de routage que PUT. La différence réside uniquement dans l'opération finale.

Après avoir localisé le nœud responsable via `find_successor`, celui-ci consulte son stockage local pour récupérer la valeur associée à la clé. Si la clé est présente, la valeur est renvoyée au client. Sinon, une réponse d'absence est transmise au client.

La latence globale d'une requête GET dépend principalement du nombre de sauts nécessaires pour atteindre le nœud responsable, ce qui reste logarithmique.

Requête DELETE

Le traitement d'une requête DELETE(key) suit le même mécanisme de localisation que les opérations PUT et GET. Le nœud qui reçoit la requête commence par calculer l'identifiant associé à la clé via la fonction de hachage, puis exécute la procédure `find_successor` afin de déterminer le nœud responsable. Une fois ce nœud atteint, la suppression est effectuée localement dans le stockage clé-valeur. Si un mécanisme de réplication est implémenté, la suppression doit également être propagée aux nœuds répliqués afin de maintenir la cohérence du système. Une confirmation est ensuite renvoyée au client. Comme pour les autres opérations, la complexité de la localisation reste logarithmique, et la suppression ne nécessite aucune coordination globale en dehors du routage vers le nœud responsable.

5.2.3 Conclusion sur l'architecture Chord

L'architecture que nous mettons en place repose sur une séparation entre le mécanisme de localisation distribué et la couche applicative de stockage. Cette structuration nous permet de construire un système scalable et entièrement décentralisé, dans lequel chaque nœud participe à la fois au routage et au stockage des données.

Cependant, si l'overlay garantit une localisation efficace des clés, la robustesse réelle du système dépendra des choix que nous ferons au niveau applicatif, notamment en matière de réplication, de gestion des pannes et de cohérence. La stabilité dynamique de l'anneau, en particulier en présence de churn, influencera directement les performances et la fiabilité du système.

Nous considérons ainsi cette architecture comme une base solide et extensible, dont le comportement effectif dépendra de la rigueur de notre implémentation et des mécanismes complémentaires que nous déciderons d'y intégrer.

5.3 Architecture générale basée sur Pastry

5.3.1 Couche overlay et couche applicative dans Pastry

Comme pour Chord, il est essentiel de distinguer deux niveaux fonctionnels.

La couche overlay correspond ici au protocole Pastry lui-même. Elle est responsable de la gestion des identifiants, du maintien des trois structures de routage et du routage progressif des messages vers le nœud cible.

Le routage s'effectue en augmentant à chaque étape la longueur du préfixe commun entre l'identifiant du nœud courant et celui de la clé recherchée. Si la clé se situe dans l'intervalle couvert par le Leaf Set, la résolution est immédiate. Sinon, la Routing Table est utilisée pour effectuer un saut vers un

nœud partageant un préfixe plus long.

La couche applicative, quant à elle, implémente le stockage clé-valeur sur le nœud responsable. Comme dans Chord, Pastry ne définit pas directement la politique de stockage : il fournit uniquement un mécanisme de localisation distribuée.

5.3.2 Cycle de vie d'une requête dans Pastry

Requête PUT

Lorsqu'un client envoie une requête PUT(key, value), celle-ci est transmise à un nœud d'entrée arbitraire du réseau.

Le nœud calcule l'identifiant de la clé via la fonction de hachage. Il compare ensuite cet identifiant à son Leaf Set. Si la clé se trouve dans l'intervalle couvert, il route directement vers le nœud numériquement le plus proche. Sinon, il consulte sa Routing Table afin de trouver un nœud partageant un préfixe plus long avec la clé.

Le message progresse ainsi de proche en proche. Chaque saut augmente la longueur du préfixe commun. Ce mécanisme garantit une complexité en $O(\log N)$.

Une fois le nœud le plus proche atteint, la paire (clé, valeur) est stockée localement. Comme pour Chord, un mécanisme de réplication peut être appliqué sur les nœuds du Leaf Set afin d'assurer la tolérance aux pannes.

Une confirmation est renvoyée au client.

Requête GET

La requête GET(key) suit exactement le même processus de routage que PUT.

Une fois le nœud responsable atteint, celui-ci consulte son stockage local. Si la clé existe, la valeur est renvoyée au client. Sinon, une réponse négative est envoyée.

La latence dépend du nombre de sauts nécessaires pour aligner progressivement les préfixes, ce qui reste en $O(\log N)$.

Requête DELETE

Le traitement de DELETE(key) reprend le même mécanisme de localisation.

Après calcul de l'identifiant, le message est routé selon la logique Leaf Set et Routing Table. Une fois le nœud cible atteint, la clé est supprimée du stockage local. Si une réplication existe, la suppression est propagée aux réplicas du Leaf Set.

5.3.3 Conclusion sur l'architecture Pastry

L'architecture Pastry repose sur un routage par préfixes qui permet d'atteindre efficacement le nœud le plus proche d'une clé tout en tenant compte de la proximité réseau. Cette approche garantit une complexité logarithmique et peut améliorer la latence réelle des communications.

Cependant, la maintenance simultanée de plusieurs structures de routage (Routing Table, Leaf Set, Neighbor Set) rend le protocole plus complexe que Chord et plus sensible aux phénomènes de churn. L'efficacité globale du système dépendra donc directement de notre implémentation.

6 Première implémentation triviale d'un DHT

L'implémentation actuelle repose sur une architecture décentralisée où chaque unité logicielle agit de manière autonome. Le système évolue suivant une progression logique, partant de l'entité de calcul jusqu'à la gestion fine de la circulation de l'information.

6.1 L'Entité de Base : Du Nœud au Thread

Le choix fondamental de conception est de représenter chaque nœud par un `NodeThread`. Dans ce modèle, le multi-threading simule l'isolation des machines d'un réseau distribué. Comme l'illustre la Figure 10, chaque nœud possède son propre cycle de vie (`run()`) et ses ressources internes.

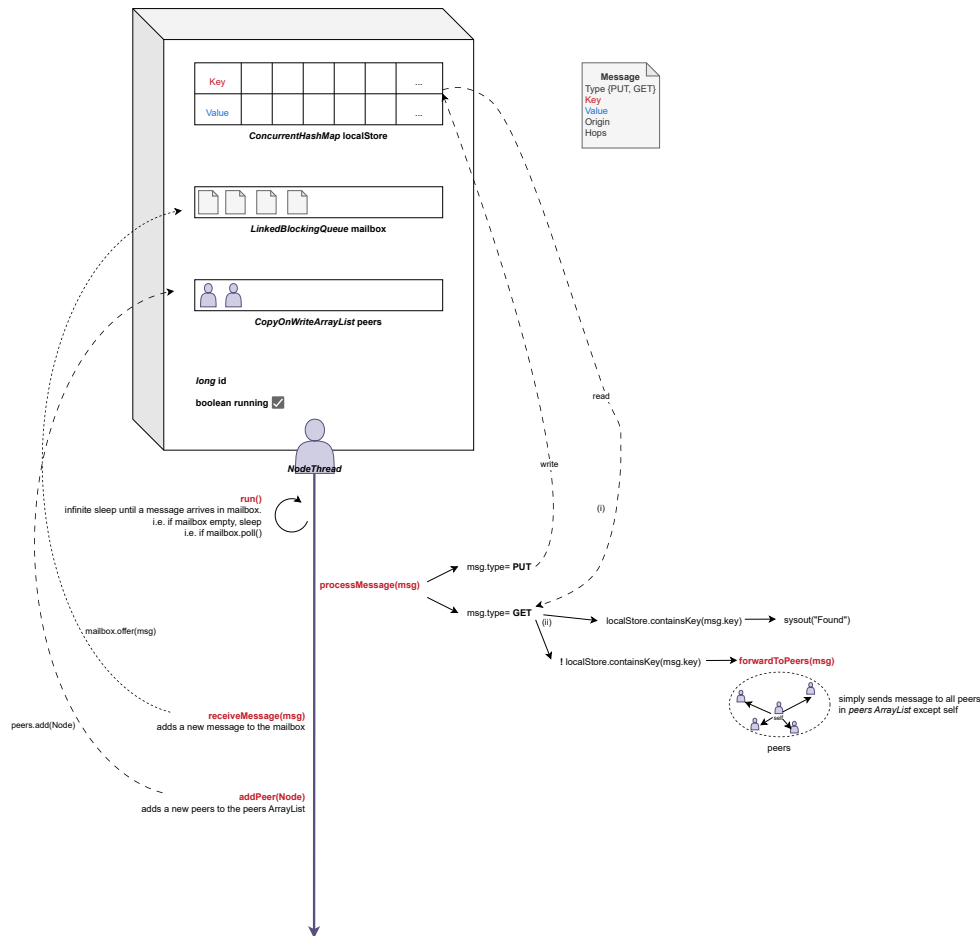


FIGURE 10 – **Architecture interne et fonctionnement du `NodeThread`.** Ce diagramme détaille la composition d'un nœud. La flèche verticale symbolise le thread actif, tandis que la boîte 3D représente l'objet `Node` avec son état (`id`, `running`) et ses structures (`peers`, `localStore`). La structure `Message` est dessinée en gris pour montrer qu'elle est stockée dans la `mailbox`. Les mots clés `Key` et `Value` sont mis en évidence pour montrer le transfert d'information du message vers le stockage local. Les fonctions principales sont aussi décrites : `run()` maintient le nœud actif, `receiveMessage()` ajoute les requêtes à la file, et `processMessage()` traite la logique métier.

Pour garantir la stabilité de ce modèle face aux accès concurrents, nous choisissons des structures de données spécifiques :

- **Le Stockage (`localStore`)** : L'utilisation d'une `ConcurrentHashMap` est indispensable. Elle permet à un thread de lire une donnée (`GET`) pendant qu'un autre message de type `PUT` met à jour la table, évitant ainsi les corruptions de données.

- **La Liste de Voisinage (peers)** : Implémentée via une `CopyOnWriteArrayList`, elle permet de parcourir les voisins en toute sécurité. Cette structure est optimale car les voisins changent peu après la phase de découverte, mais sont consultés à chaque transfert de message.

6.2 Le Mécanisme de Communication : La Mailbox

Une fois les nœuds actifs, ils communiquent via une `mailbox` implémentée par une `LinkedBlockingQueue`. Ce mécanisme permet de découpler l'envoi de la réception. Le choix de la méthode d'insertion est ici stratégique :

- **L'avantage de `offer()`** : Contrairement à `add()` ou `put()`, la fonction `receiveMessage(msg)` utilise `mailbox.offer(msg)`. C'est une tentative non-bloquante qui ajoute le message à la file sans interrompre l'expéditeur. Cela prévient les *deadlocks* (blocages mutuels) où deux nœuds s'attendraient indéfiniment.

6.3 Évolution du Message : Du simple "Sender" à l'"Origin"

La gestion des messages constitue le cœur du routage. L'analyse a montré qu'en relayant un message, le nœud receveur perdait la trace de l'initiateur, rendant la réponse impossible.

Comme l'illustre la Figure 11, nous sommes passés d'une structure simple à une structure permettant le suivi :

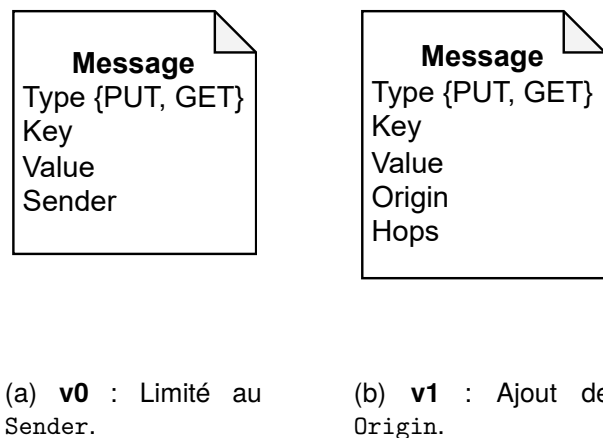


FIGURE 11 – Comparaison de l'évolution de la structure du Message.

Dans la version initiale (**v0**), le message ne contenait que l'expéditeur immédiat. La version actuelle (**v1**) remplace ce champ par l'`Origin`. En conservant cette référence fixe, le message garde son identité tout au long de son parcours.

6.4 Mesure du parcours et topologie du système

Le `DHTSystem` orchestre le réseau via deux étapes clés : l'ajout de nœuds (`addNode`) et la création de liens bidirectionnels (`buildDiscoveryChain`). Comme le montre la Figure 12, cette chaîne permet la propagation des messages de proche en proche. Pour diagnostiquer l'efficacité de ce routage, nous intégrons un compteur de `Hops`. À chaque transfert vers un pair, ce compteur s'incrémente, permettant de détecter les inefficacités ou les recherches trop longues.

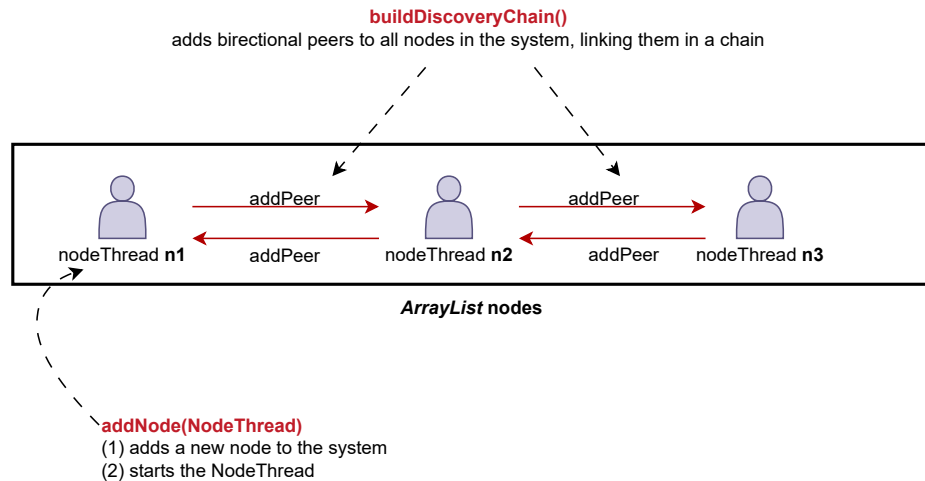


FIGURE 12 – **Topologie et initialisation du réseau via DHTSystem.** Ce schéma illustre l'organisation des nœuds stockés dans une `ArrayList`. En bas, la fonction `addNode()` est indiquée comme responsable de l'ajout d'un nouveau nœud et du démarrage de son thread. En haut, la fonction `buildDiscoveryChain()` est mise en évidence : elle parcourt la liste pour établir des connexions bidirectionnelles entre les nœuds voisins. Ces connexions sont matérialisées par les flèches rouges `addPeer`, transformant la simple liste en une chaîne de propagation opérationnelle.

6.5 Analyse des Erreurs et Limites de la V0

Cette première implémentation, bien que fonctionnelle pour les scénarios nominaux, présente des failles critiques identifiées lors des tests.

6.6 Erreurs majeures rencontrées

- **Boucles de recherche infinies** : Si une clé demandée n'existe pas dans le système (ex: recherche de "Shape"), le message est transféré de nœud en nœud sans jamais s'arrêter. Les threads continuent de saturer la file d'attente de leurs voisins indéfiniment.
- **Problème de Flooding** : Actuellement, un nœud propage le message à *tous* ses pairs. Dans un réseau plus large, cela provoquerait une explosion exponentielle du nombre de messages, paralysant rapidement les ressources CPU et mémoire.
- **Absence de mécanisme de retour** : Lorsqu'un nœud trouve une donnée, il se contente d'afficher "FOUND" dans la console. L'origine de la requête n'est jamais notifiée formellement par le biais d'un message retour.

6.7 Points d'améliorations pour la V1

Pour stabiliser le système, les évolutions suivantes sont prioritaires :

1. **Time To Live (TTL)** : Intégrer une limite de sauts au sein de la classe `Message`. Une fois le compteur à zéro, le message est supprimé, stoppant ainsi les recherches infructueuses.
2. **Type RESPONSE** : Créer un nouveau type de message pour permettre l'acheminement de la valeur trouvée directement vers l'initiateur (`Origin`).
3. **Mémorisation des requêtes** : Faire en sorte que chaque nœud garde en mémoire les identifiants des messages déjà traités pour éviter de relayer plusieurs fois la même requête.